



Data Structures

CPT102 – Lecture 12
Erick Purwanto



Xi'an Jiaotong-Liverpool University

西交利物浦大學

CPT102 Data Structures

Lecture 12

Binary Search Trees

Welcome!

- Welcome to Lecture 12 !
- In this lecture we are going to
 - learn about map / symbol tables / dictionaries
 - implement it using a binary search tree
 - learn about binary search trees
 - implement search (get) and insert (put)
 - learn about balanced search trees
 - implement it conceptually with 2-3 trees
 - implement 2-3 trees with red-black trees

Map



- Key-value pairs, also known as symbol tables, dictionary, associative arrays
 - **insert** a value with specified key
 - given a key, **search** for the corresponding value
 - for example: DNS search, given domain name, find corresponding IP address

domain name	IP address
www.princeton.edu	128.112.128.15
www.yale.edu	130.132.143.21
www.harvard.edu	128.103.060.55
www.simpsons.com	209.052.165.60

↑
key

↑
value

Map in Java



- A **map** stores key/value pairs, where each **key** has an associated **value**
 - given a particular key, the map can look up the associated value *very quickly* !
- Implementation of maps are TreeMap and HashMap
 - this week, we will learn the basic ideas behind TreeMap
 - *HashMap for next week*

Map Operations in Java

- Operations in maps include:

- `map.put(key, val)`
- `map.get(key)`
- `map.containsKey(key)`
- `map.containsValue(value)`
- `map.remove(key)`
- `map.keySet()`

adding the mapping key → val

get the value for a key

test whether the map has the key

test whether the map has the value

delete a mapping

returns a set containing all keys

Map Example in Java (1)

- Suppose we have a list of strings of multiple duplicate words, such as ["a", "b", "a", "c", "b", "word", "a", "word"]
- We want to find out which words appear in the list and how many times each one appears
- Here is an algorithm NumWords using Map:
 - Create a Map<String, Integer>
 - Loop through all the strings in the list
 - Use each string as a key into the map
 - Use the Integer value to store the number of times that string has been seen
 - There are 2 cases:
 - the first time we see a word, it is not yet in the map, we add it
 - the later times we see the word, it is already in the map, we update it

Map Example in Java (2)

```
import java.util.HashMap;
import java.util.Map;

public class NumWords {
    public Map<String, Integer> wordCount(List<String> list) {
        Map<String, Integer> map = new HashMap<>();
        int num;
        for (String word : list) {
            if (!map.containsKey(word)) {
                map.put(word, 1);
            }
            else {
                num = map.get(word);
                map.put(word, num + 1);
            }
        }
        return map;
    }
}
```

the first time we've seen string word

we increment the number of word

In-Class Quiz 1

- Suppose we implement a map using **an ordered array**
 - two arrays, the elements in keys are sorted
 - what is the running time of **insert**?
 - what is the running time of **search**?

keys[]										vals[]									
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
A	C	E	H	L	M	P	R	S	X	8	4	12	5	11	9	10	3	0	7

Unordered Array

- To insert, need to shift all greater keys over (and the corresponding values)

		keys[]											vals[]									
key	value	0	1	2	3	4	5	6	7	8	9	N	0	1	2	3	4	5	6	7	8	9
S	0	S										1	0									
E	1	E	S									2	1	0								
A	2	A	E	S								3	2	1	0							
R	3	A	E	R	S							4	2	1	3	0						
C	4	A	C	E	R	S						5	2	4	1	3	0					
H	5	A	C	E	H	R	S					6	2	4	1	5	3	0				
E	6	A	C	E	H	R	S					6	2	4	6	5	3	0				
X	7	A	C	E	H	R	S	X				7	2	4	6	5	3	0	7			
A	8	A	C	E	H	R	S	X				7	8	4	6	5	3	0	7			
M	9	A	C	E	H	M	R	S	X			8	8	4	6	5	9	3	0	7		
P	10	A	C	E	H	M	P	R	S	X		9	8	4	6	5	9	10	3	0	7	
L	11	A	C	E	H	L	M	P	R	S	X	10	8	4	6	5	11	9	10	3	0	7
E	12	A	C	E	H	L	M	P	R	S	X	10	8	4	12	5	11	9	10	3	0	7
		A	C	E	H	L	M	P	R	S	X		8	4	12	5	11	9	10	3	0	7

entries in red were inserted

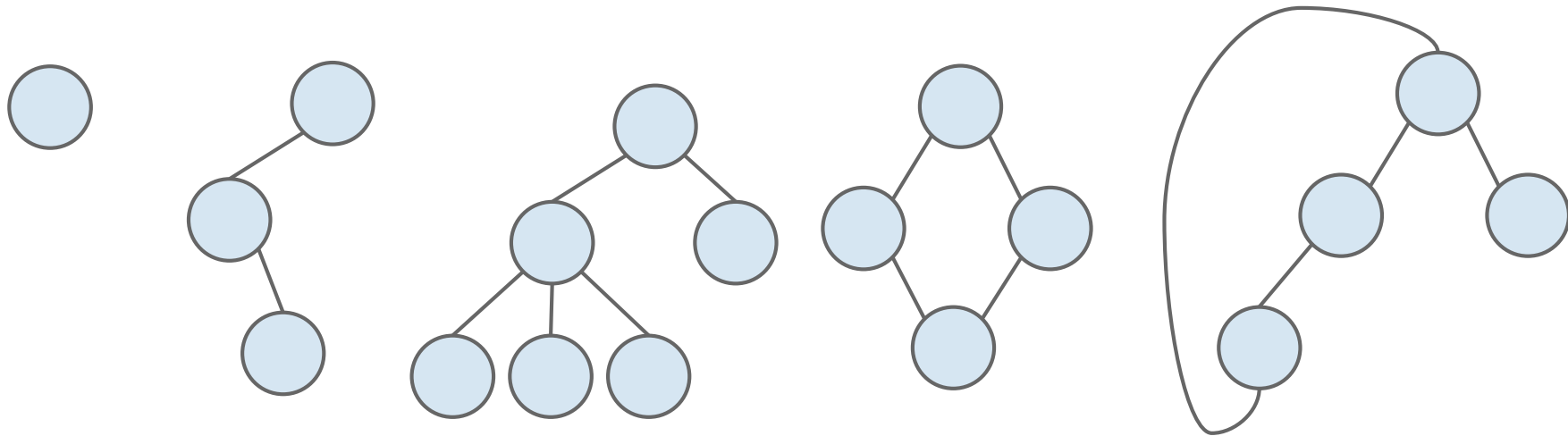
entries in gray did not move

entries in black moved to the right

circled entries are changed values

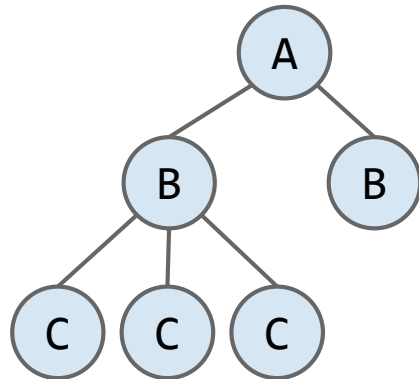
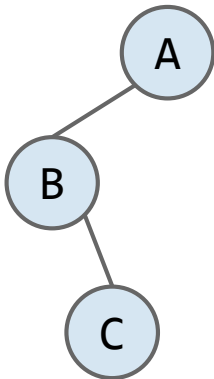
Tree

- A tree consists of:
 - a set of nodes and a set of edges that connect those nodes
 - constraint: there is **exactly one path** between any two nodes



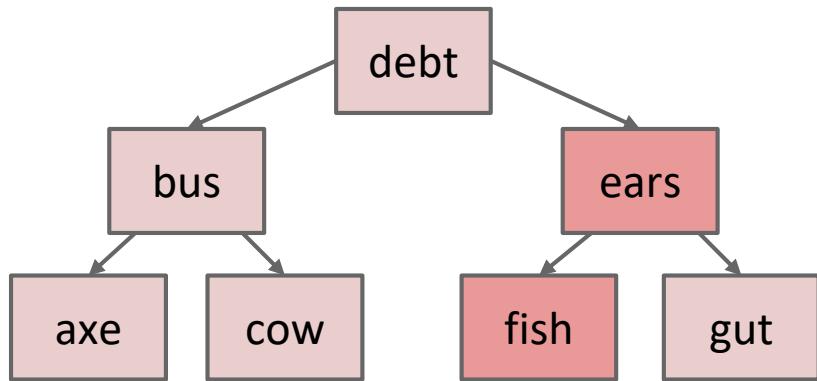
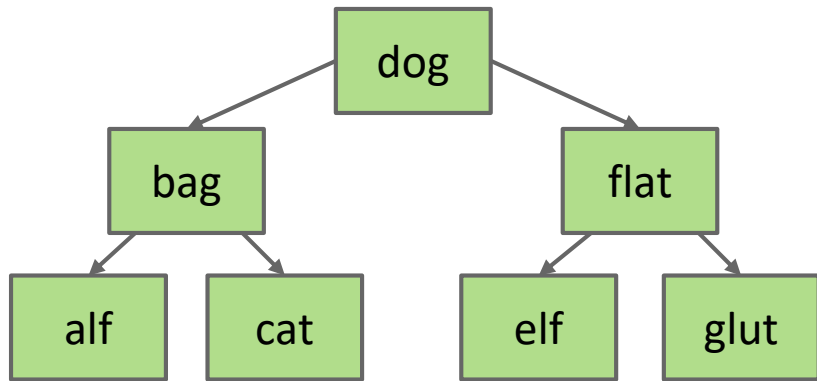
Rooted Binary Trees

- In a **rooted tree**, we call one node the root
 - every node N except the root has exactly one parent, defined as the first node on the path from N to the root
 - a node with no child is called a leaf
- In a **rooted binary tree**, every node has either 0, 1, or 2 children



Binary Search Trees

- A **binary search tree** is a rooted binary tree with the **BST property**, for every node X in the tree:
 - every key in the **left** subtree is **less** than X's key
 - every key in the **right** subtree is **greater** than X's key



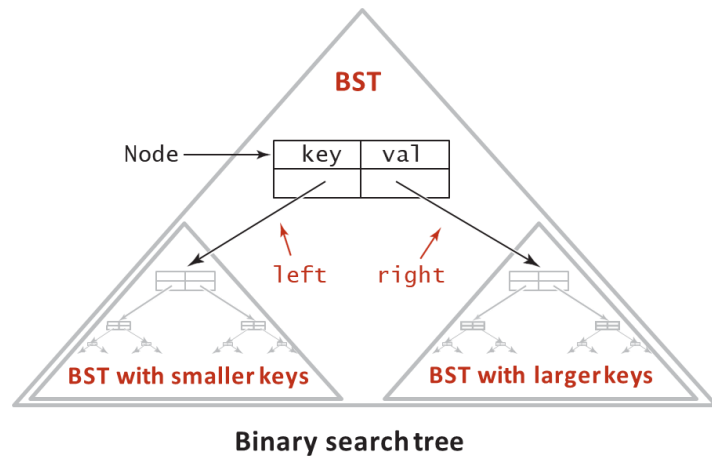
Binary Search Trees Ordering

- Ordering must be **complete**, **transitive**, and **antisymmetric**: given keys p and q :
 - exactly one of $p < q$ and $q < p$ are true
 - $p < q$ and $q < r$ imply $p < r$
 - in Java, keys must be Comparable
- **No duplicate keys** allowed!
 - what happens when an existing key with a new value associated gets inserted?

BST Node

- **BST Node** implementation:

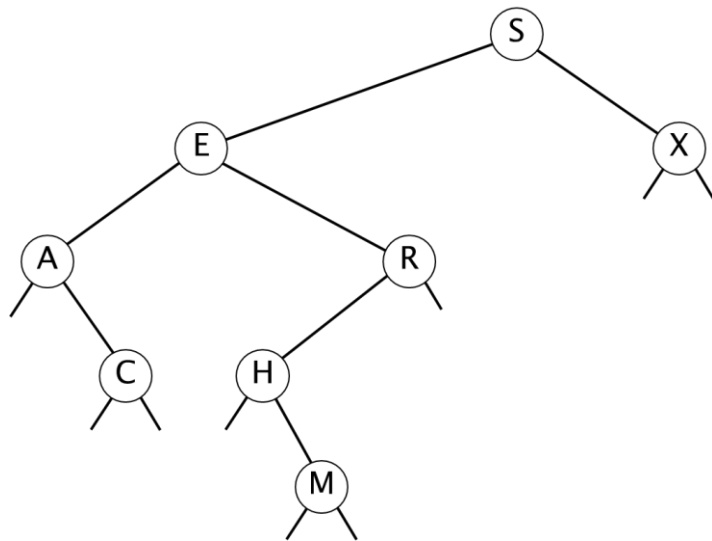
```
private class Node {  
    private Key key;  
    private Value val;  
    private Node left, right;  
    public Node(Key key, Value val) {  
        this.key = key;  
        this.val = val;  
    }  
}
```



Searching a Key in BST

- **get:** if key equals the key of node x: x.key, return the value
 - if $\text{key} < \text{x.key}$, search left subtree: x.left
 - if $\text{key} > \text{x.key}$, search right subtree: x.right

```
public Value get(Node x, Key key) {  
    if (x == null)  
        return null;  
    if (key.equals(x.key))  
        return x;  
    else if (key < x.key)  
        return get(x.left, key);  
    else  
        return get(x.right, key);  
}
```



In-Class Quiz 2

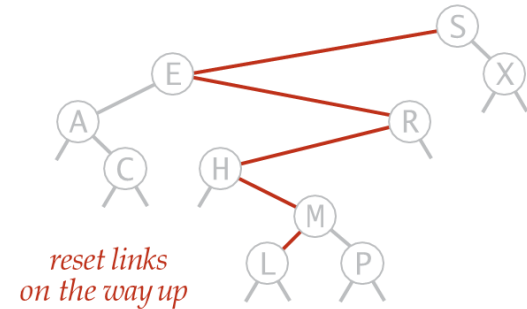
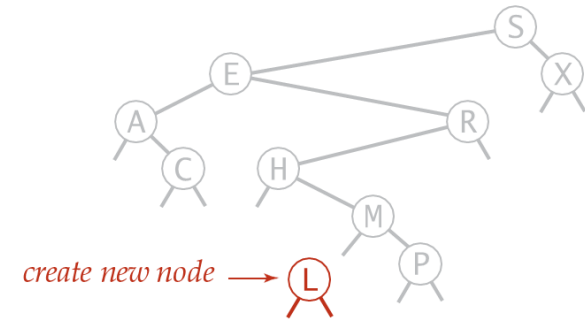
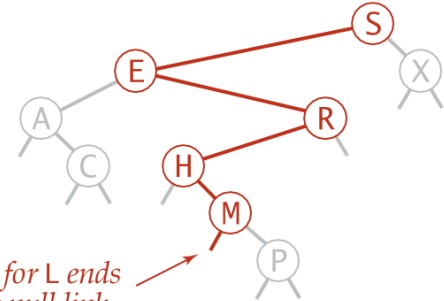
- What is the purpose of checking equality with null, and returning null?

```
public Value get(Node x, Key key) {  
    if (x == null)  
        return null;  
    if (key.equals(x.key))  
        return x;  
    else if (key < x.key)  
        return get(x.left, key);  
    else  
        return get(x.right, key);  
}
```

Inserting a New Key into BST

- **put**: associate a new value with key
- Search for key, then:
 - if key in tree, update value
 - if key not in tree, add new node

inserting L



Insert Implementation

- **put**: associate a new value with key
- Search for key, then: if key in tree, update value; if key not in tree, add new node

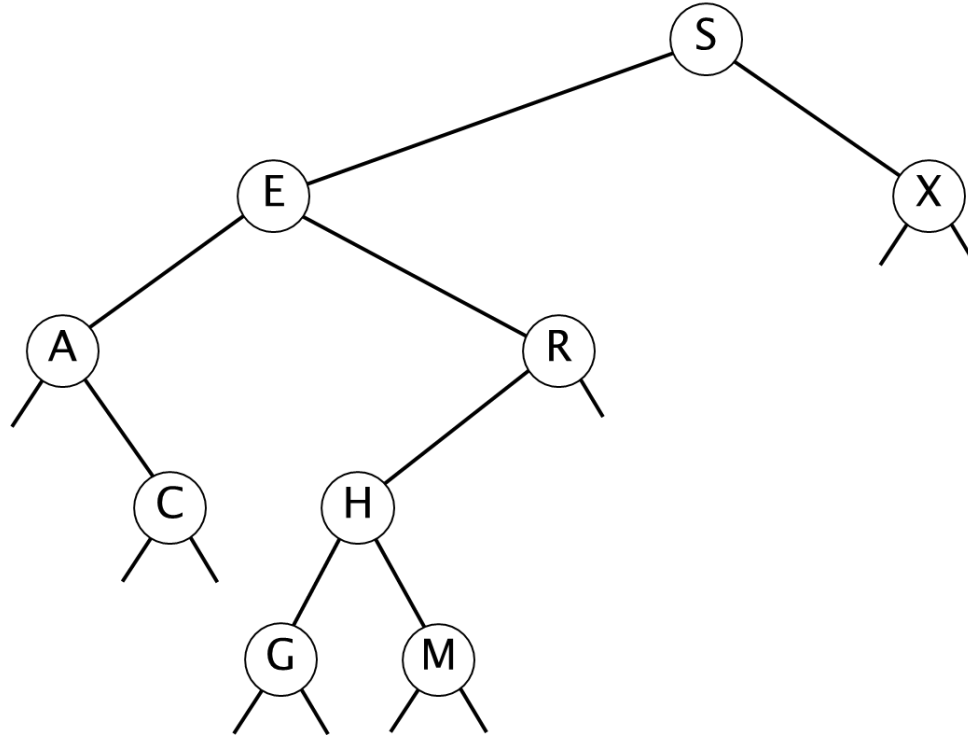
```
public void put(Key key, Value val)
{
    root = put(root, key, val);
}

private Node put(Node x, Key key, Value val)
{
    if (x == null) return new Node(key, val);
    int cmp = key.compareTo(x.key);
    if (cmp < 0)
        x.left = put(x.left, key, val);
    else if (cmp > 0)
        x.right = put(x.right, key, val);
    else if (cmp == 0)
        x.val = val;
    return x;
}
```

if the same key is found,
update the value

In-Class Quiz 3

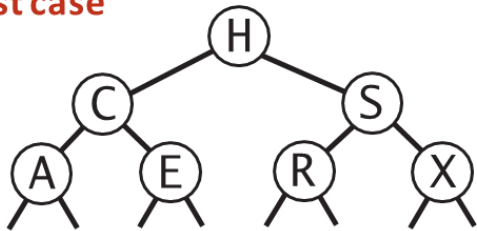
- Describe what happened when a node K is inserted into the BST:



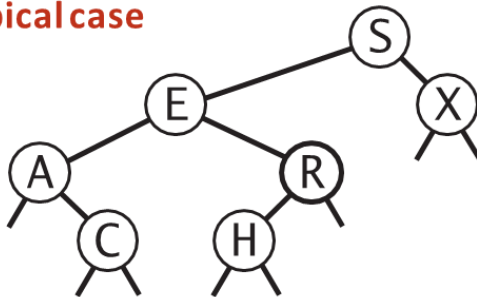
Running Time of Search and Insert

- The number of **compares** for search/insert is equal to **1 + depth of node**
 - depends on the tree shape
 - many BST corresponds to the same set of keys

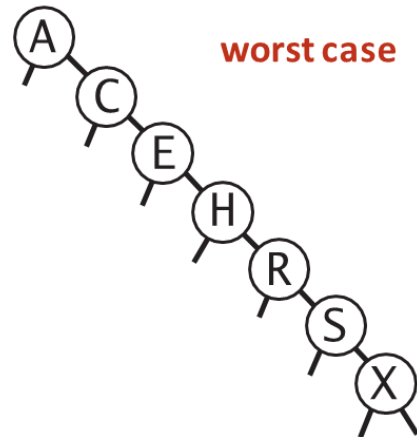
best case



typical case



worst case



- tree shape depends on the order of insertion

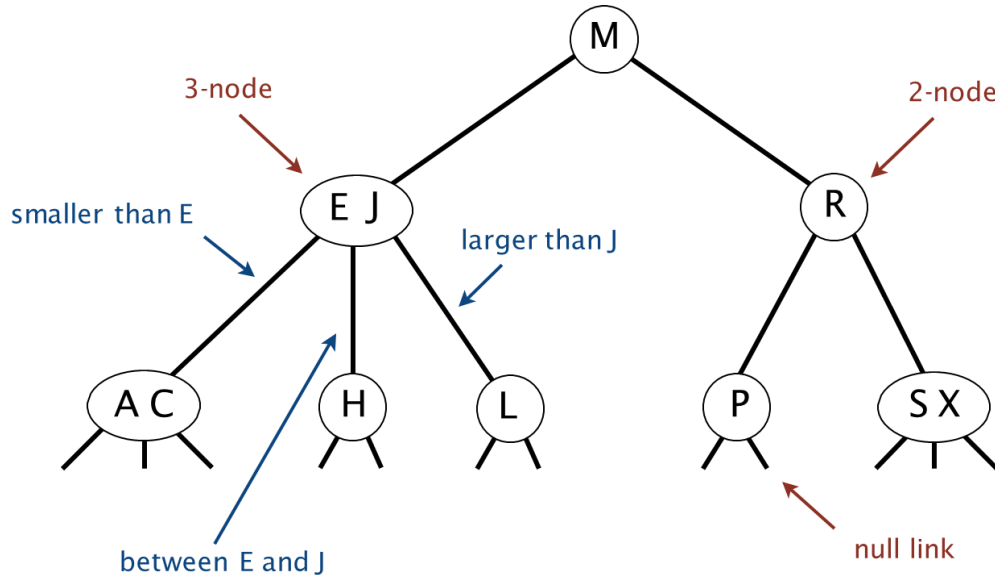
$O(N)$ for search/insert in the worst case, can we do better?

Balanced Search Trees

- Want **$O(\log n)$** guaranteed (worst-case) performance for insert and search
- Binary search trees may not be *balanced*
- Balanced search trees:
 - **2-3 trees** – see conceptually balanced
 - **left-leaning red-black trees** – to implement 2-3 trees

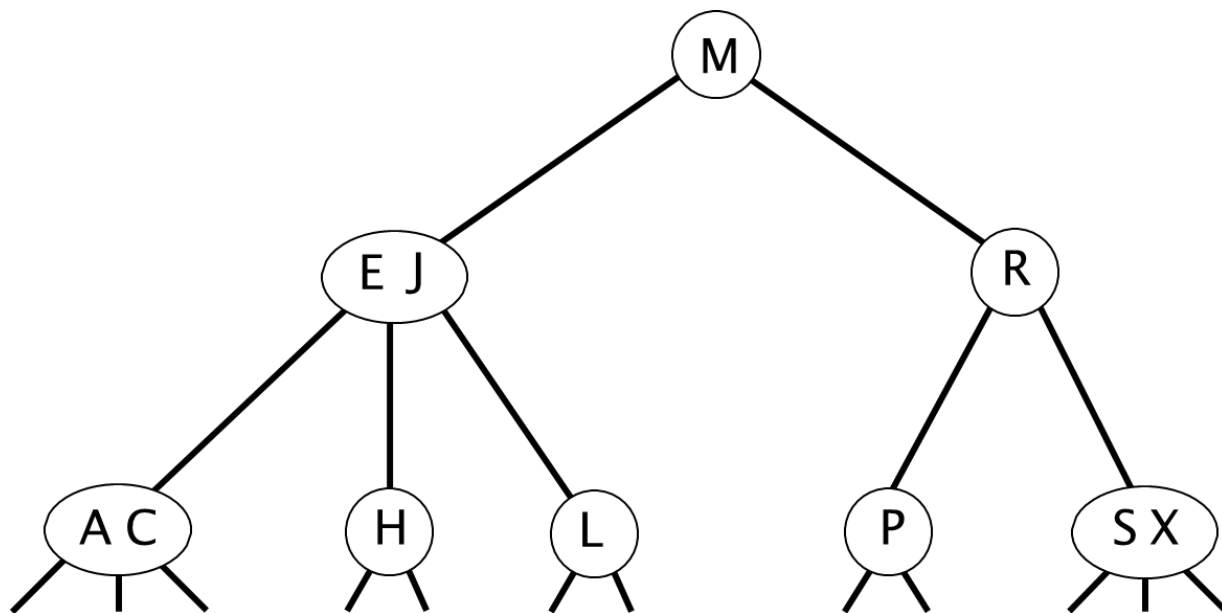
2-3 Trees

- Allow 1 or 2 keys per node
 - **2-node**: one key, two children; and **3-node**: two keys, three children
- **Symmetric order**: inorder traversal yields keys in ascending order
- **Perfect balance**: every path from root to null link has same length



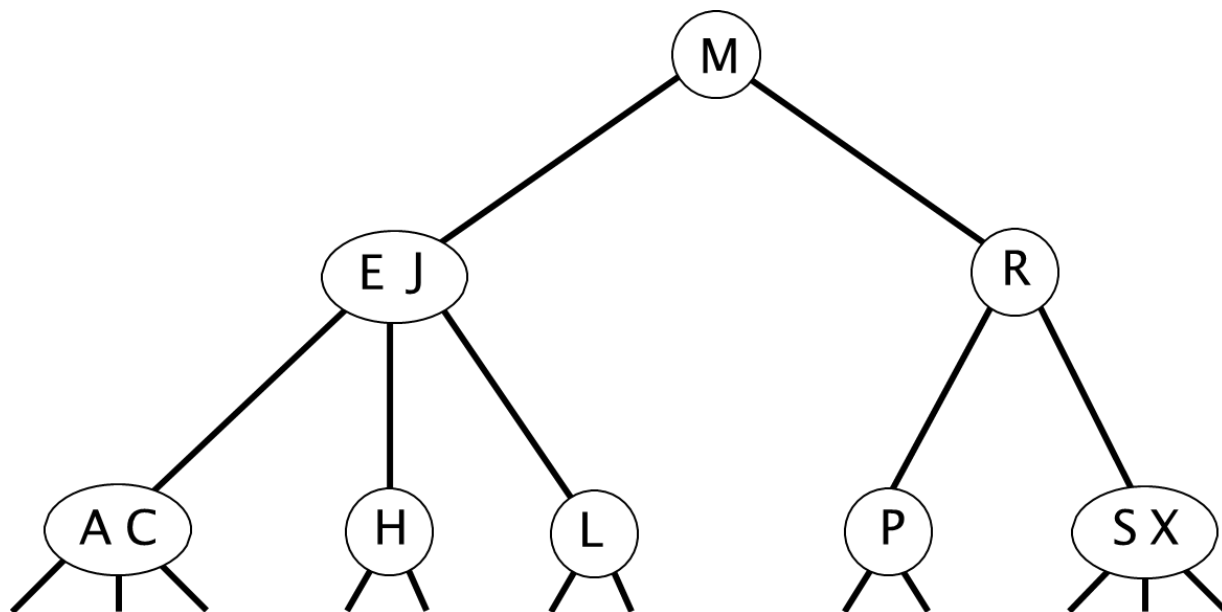
2-3 Tree Search

- Compare search key against keys in node
- Find interval containing search key
- Follow associated link (recursively)



In-Class Quiz 4

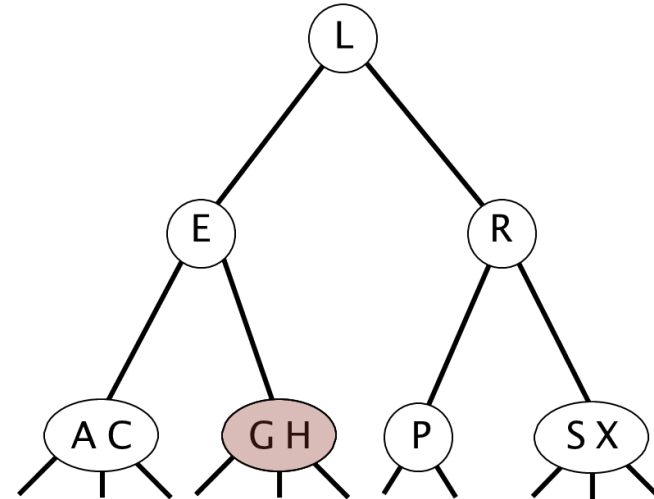
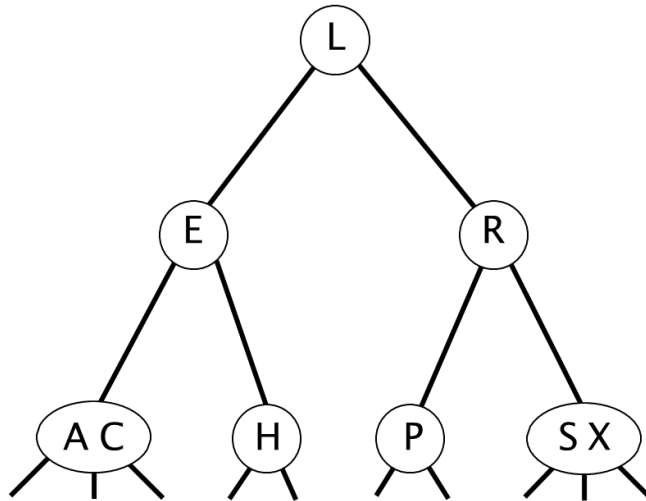
- Suppose you want to search for key T
- Describe the search process on the following 2-3 tree:



2-3 Tree Insert (1)

- Case 1: Insertion into a 2-node at bottom
 - add new key to 2-node to create a 3-node

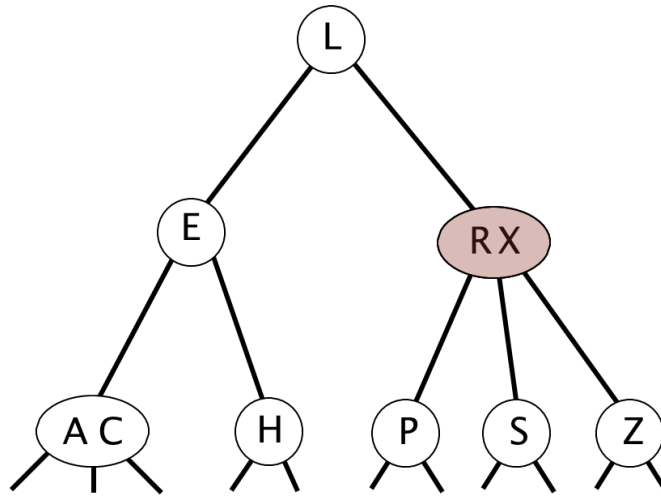
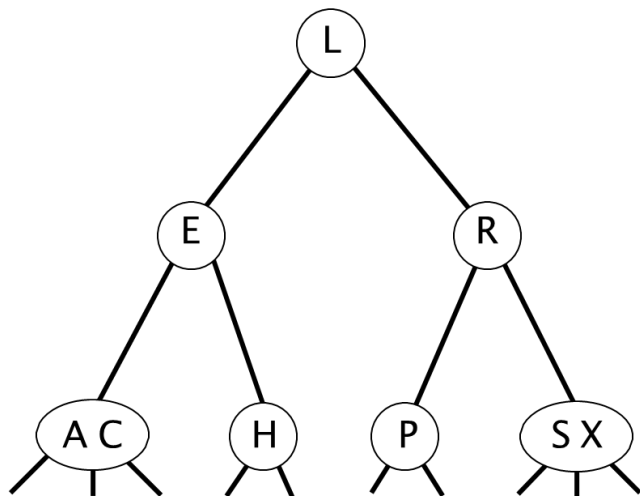
insert G



2-3 Tree Insert (2)

- Case 2: Insertion into a 3-node at bottom
 - Add new key to 3-node to create **temporary 4-node**
 - Move middle key in 4-node into parent; Repeat up the tree, as necessary
 - If you reach the root and it's a 4-node, **split** it into **three** 2-nodes

insert Z

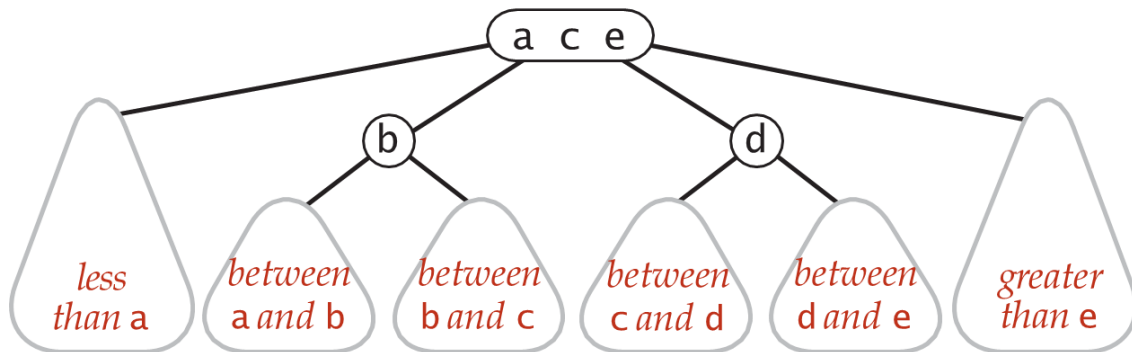
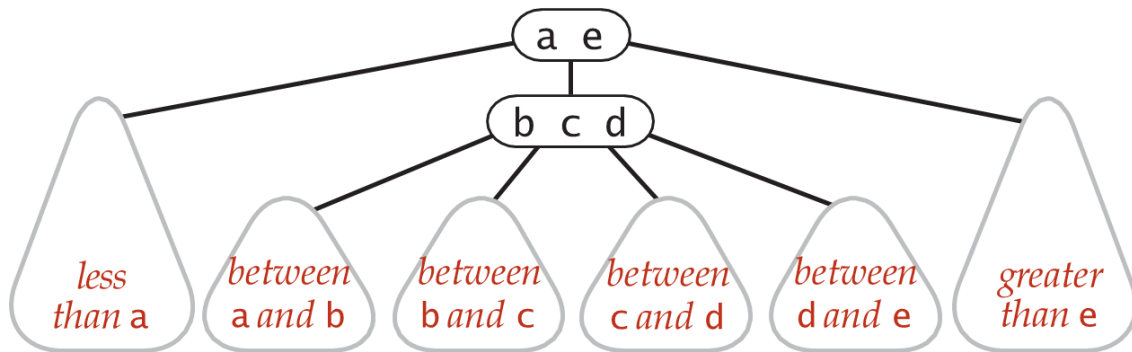


In-Class Quiz 5

- Suppose that you are inserting a **new key** into a **2-3 tree**
- Under what scenario must the **height** of the 2-3 tree *increase by one*?

Local Transformations in 2-3 Tree

- Splitting a 4-node is a **local** transformation: **constant** number of operations



Global Properties in 2-3 Tree

- Invariants: maintains symmetric order and perfect balance
- Proof: each transformation maintains symmetric order and perfect balance

root



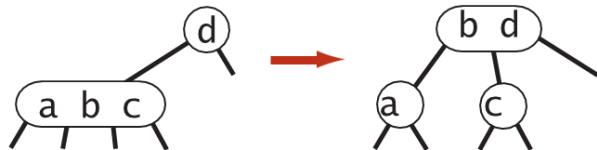
parent is a 3-node

left



parent is a 2-node

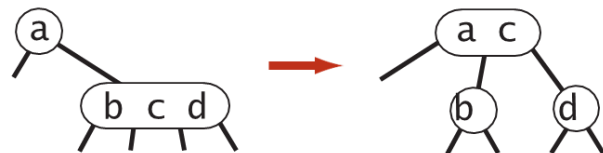
left



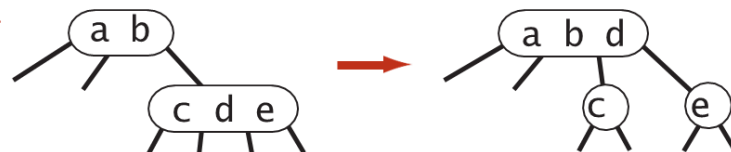
middle



right

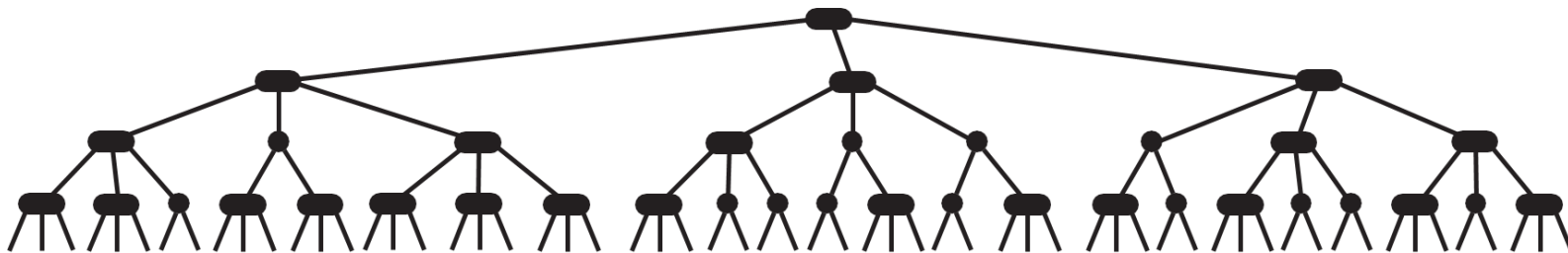


right



2-3 Tree Running Time

- Perfect balance: every path from root to null link has same length



- Worst case tree height is $O(\log n)$
 - best case: $\log_3 n$

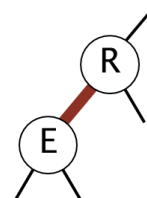
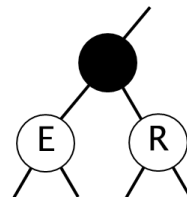
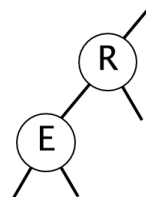
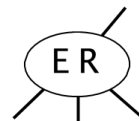
guaranteed $O(\log n)$
for search and insert!

2-3 Tree Implementation (?)

- Direct implementation is complicated, because:
 - maintaining multiple node types is cumbersome
 - need multiple compares to move down tree
 - need to move back up the tree to split 4-nodes
 - large number of cases for splitting
- A better way: implement using **red-black trees**

Implementing 2-3 Trees with Binary Trees

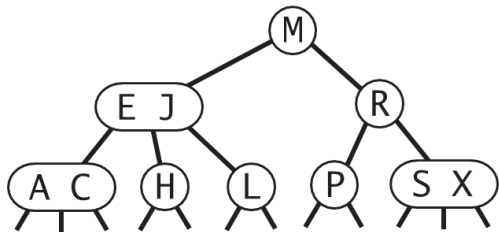
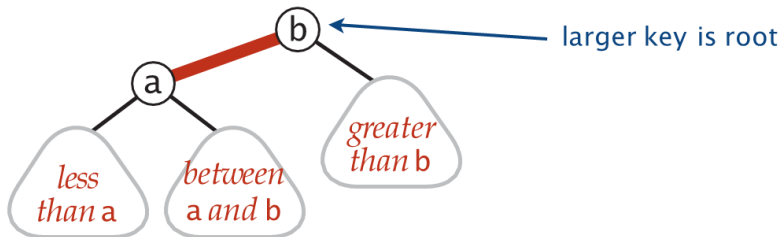
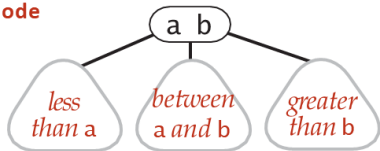
- How to represent a 3-node?
- Approach 1: regular BST
 - no way to tell a 3-node from a 2-node
 - cannot map from BST back to 2-3 tree
- Approach 2: regular BST with "glue" nodes
 - wastes space, wasted link
 - code probably messy
- Approach 3: regular BST with **red "glue" links**
 - widely used in practice
 - arbitrary restriction: red links *lean left*



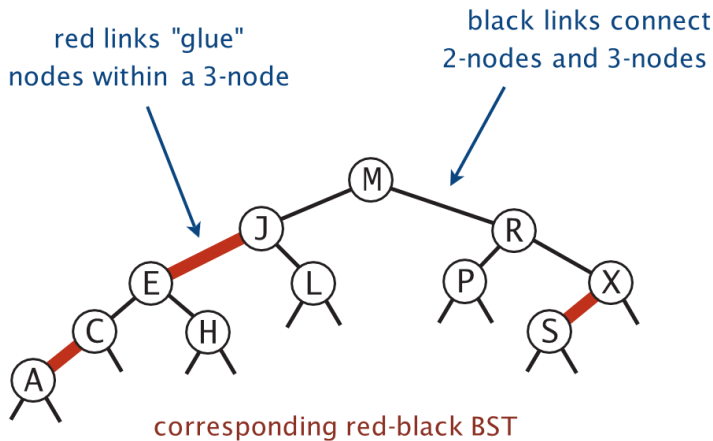
Left-Leaning Red-Black BSTs

- Represent 2–3 tree as a BST, use *internal* left-leaning links as *glue* for 3-nodes

3-node

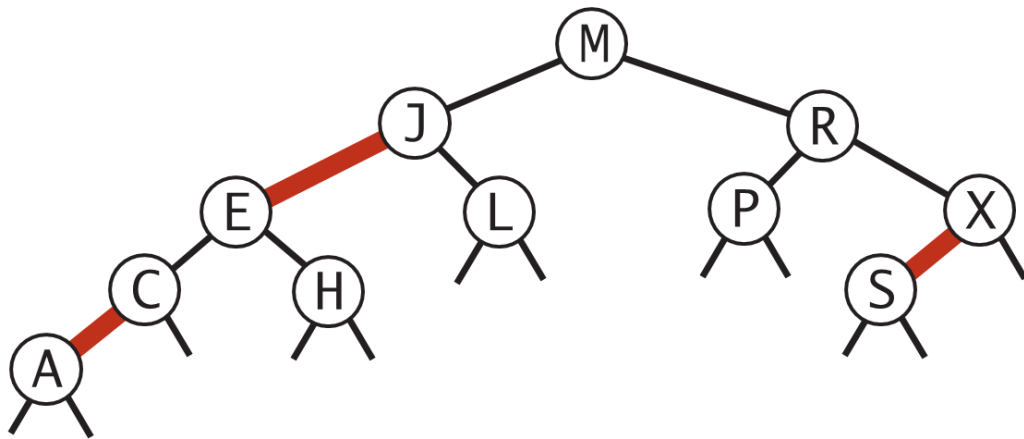


2-3 tree



Left-Leaning Red-Black BSTs Definition

- A binary search tree such that:
 - no node has two red links connected to it
 - **perfect black balance:** every path from root to null link has the same number of black links
 - red links lean left

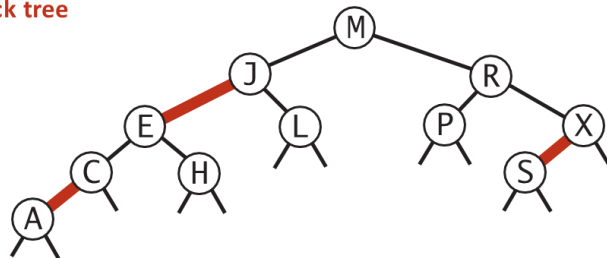


need to fix these three properties if ever violated

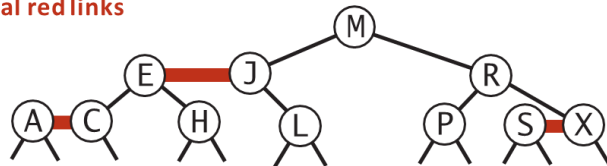
1-1 Correspondence

- Key property: 1–1 correspondence between 2–3 and LLRB trees
 - if we can implement it correctly, search and insert will be $O(\log n)$

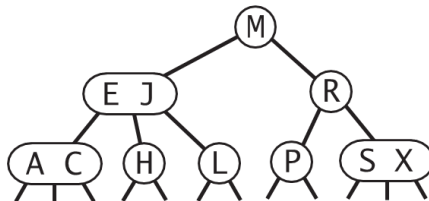
red-black tree



horizontal red links



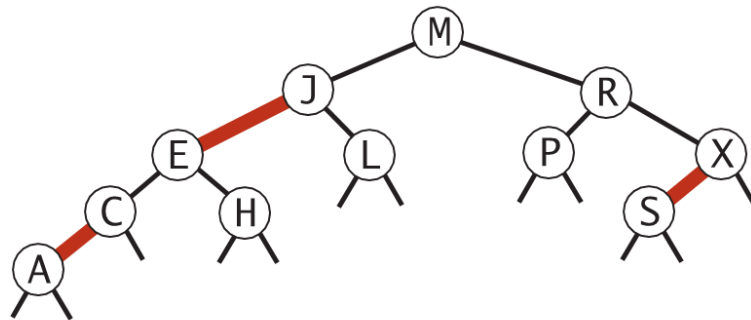
2-3 tree



LLRB Trees Search

- Search is the same as for elementary BST by ignoring color

```
public Val get(Key key)
{
    Node x = root;
    while (x != null)
    {
        int cmp = key.compareTo(x.key);
        if      (cmp < 0) x = x.left;
        else if (cmp > 0) x = x.right;
        else if (cmp == 0) return x.val;
    }
    return null;
}
```



LLRB Trees Node

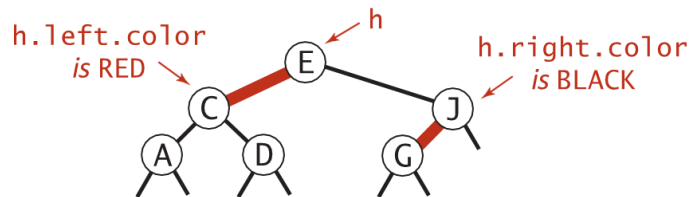
- Each node is pointed to by precisely one link (from its parent)
 - we encode color of links in nodes

```
private static final boolean RED    = true;  
private static final boolean BLACK = false;
```

```
private class Node  
{  
    Key key;  
    Value val;  
    Node left, right;  
    boolean color; // color of parent link  
}
```

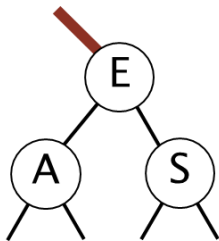
```
private boolean isRed(Node x)  
{  
    if (x == null) return false;  
    return x.color == RED;  
}
```

null links are black

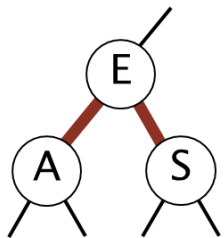


LLRB Trees Insertion Overview

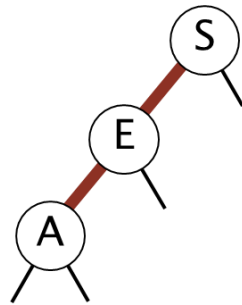
- Maintain 1-1 correspondence with 2-3 trees
 - during internal operations, maintain symmetric order and perfect black balance



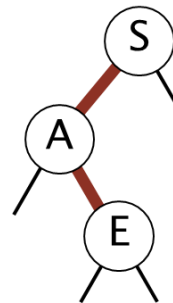
right-leaning
red link



two red children
(a temporary 4-node)



left-left red
(a temporary 4-node)



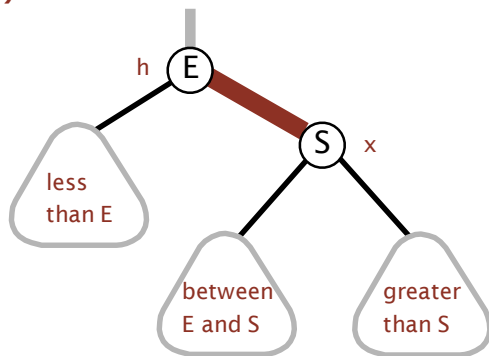
left-right red
(a temporary 4-node)

- by applying LLRB tree operations: **rotation** and **color flip**

Elementary red-black BST operations

Left rotation. Orient a (temporarily) right-leaning red link to lean left.

rotate E left
(before)

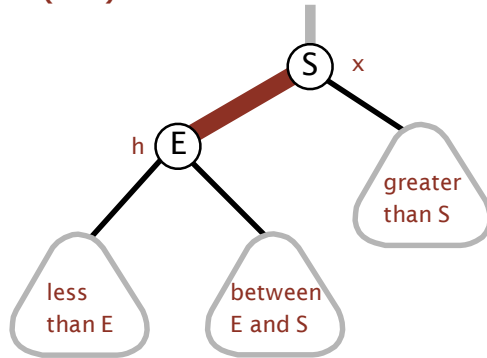


```
private Node rotateLeft(Node h)
{
    assert isRed(h.right);
    Node x = h.right;
    h.right = x.left;
    x.left = h;
    x.color = h.color;
    h.color = RED;
    return x;
}
```


Elementary red-black BST operations

Left rotation. Orient a (temporarily) right-leaning red link to lean left.

rotate E left
(after)

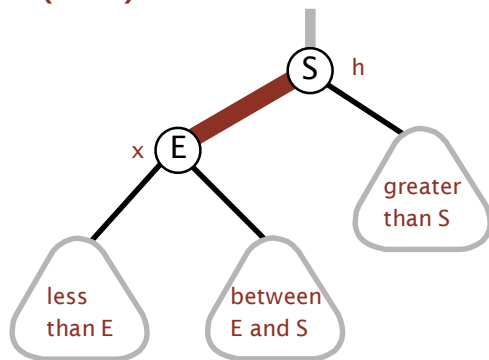


```
private Node rotateLeft(Node h)
{
    assert isRed(h.right);
    Node x = h.right;
    h.right = x.left;
    x.left = h;
    x.color = h.color;
    h.color = RED;
    return x;
}
```

Elementary red-black BST operations

Right rotation. Orient a left-leaning red link to (temporarily) lean right.

rotate S right
(before)

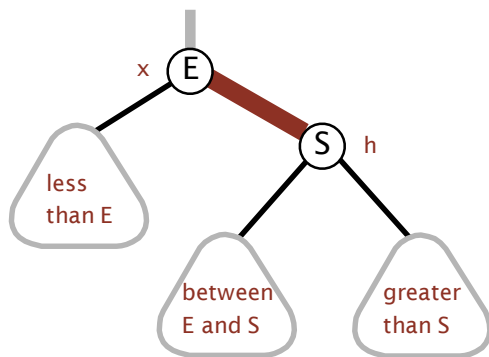


```
private Node rotateRight(Node h)
{
    assert isRed(h.left);
    Node x = h.left;
    h.left = x.right;
    x.right = h;
    x.color = h.color;
    h.color = RED;
    return x;
}
```

Elementary red-black BST operations

Right rotation. Orient a left-leaning red link to (temporarily) lean right.

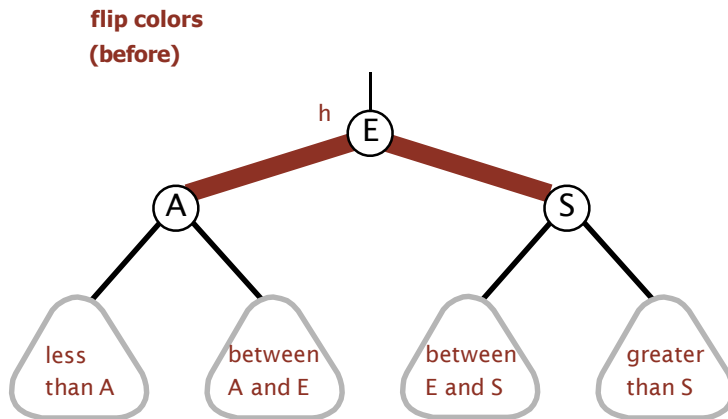
rotate S right
(after)



```
private Node rotateRight(Node h)
{
    assert isRed(h.left);
    Node x = h.left;
    h.left = x.right;
    x.right = h;
    x.color = h.color;
    h.color = RED;
    return x;
}
```

Elementary red-black BST operations

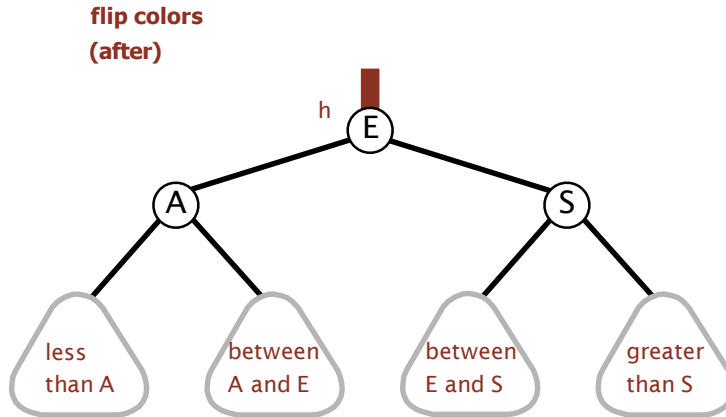
Color flip. Recolor to split a (temporary) 4-node.



```
private void flipColors(Node h)
{
    assert !isRed(h);
    assert isRed(h.left);
    assert isRed(h.right);
    h.color = RED;
    h.left.color = BLACK;
    h.right.color = BLACK;
}
```

Elementary red-black BST operations

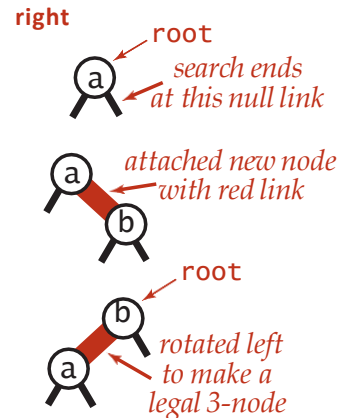
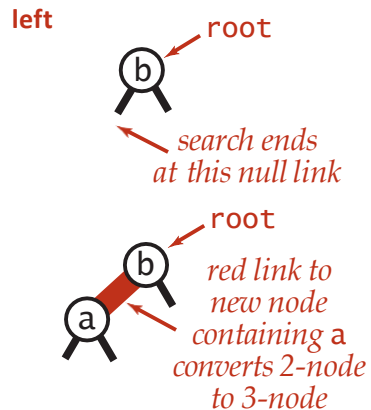
Color flip. Recolor to split a (temporary) 4-node.



```
private void flipColors(Node h)
{
    assert !isRed(h);
    assert isRed(h.left);
    assert isRed(h.right);
    h.color = RED;
    h.left.color = BLACK;
    h.right.color = BLACK;
}
```

Insertion in a LLRB tree

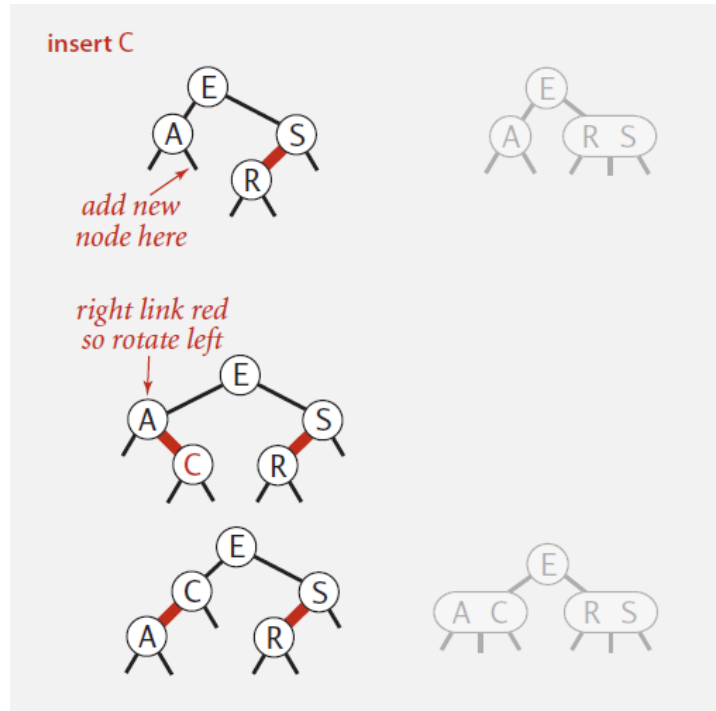
Warmup 1. Insert into a tree with exactly 1 node.



Insertion in a LLRB tree

Case 1. Insert into a 2-node at the bottom.

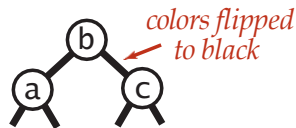
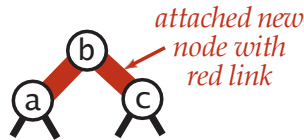
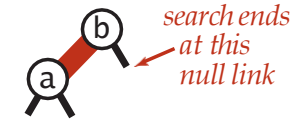
- Do standard BST insert; color new link red. ← to maintain symmetric order and perfect black balance
- If new red link is a right link, rotate left. ← to fix color invariants



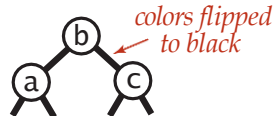
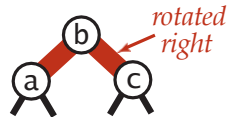
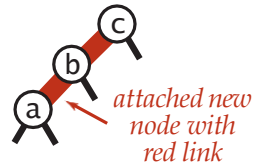
Insertion in a LLRB tree

Warmup 2. Insert into a tree with exactly 2 nodes.

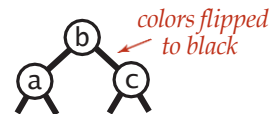
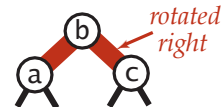
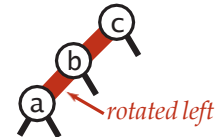
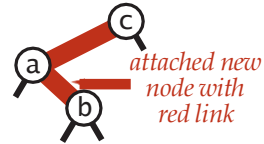
larger



smaller



between



Insertion in a LLRB tree

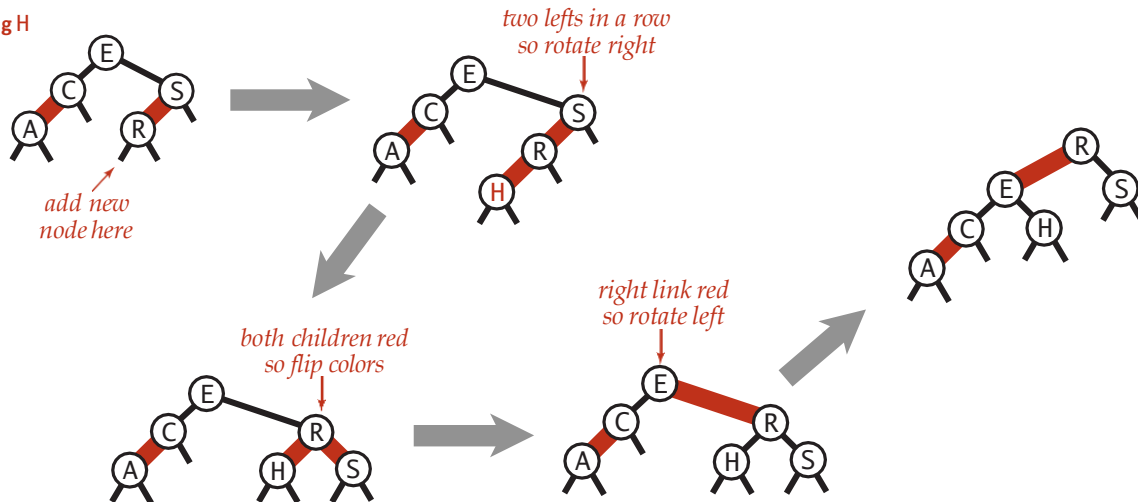
Case 2. Insert into a 3-node at the bottom.

- Do standard BST insert; color new link red.
- Rotate to balance the 4-node (if needed).
- Flip colors to pass red link up one level.
- Rotate to make lean left (if needed).

to maintain symmetric order
and perfect black balance

to fix color invariants

inserting H



Insertion in a LLRB tree: passing red links up the tree

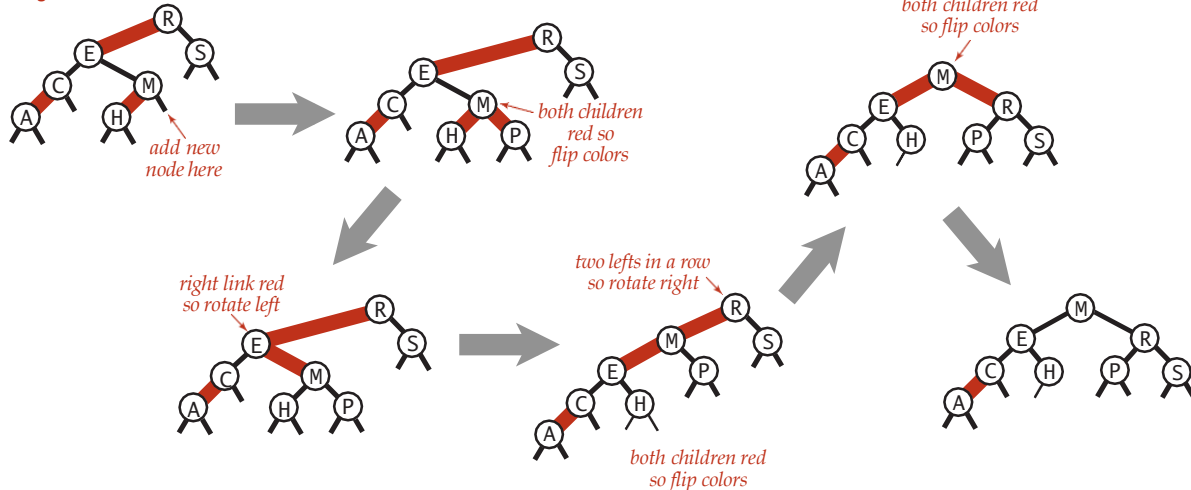
Case 2. Insert into a 3-node at the bottom.

- Do standard BST insert; color new link red.
- Rotate to balance the 4-node (if needed).
- Flip colors to pass red link up one level.
- Rotate to make lean left (if needed).
- Repeat case 1 or case 2 up the tree (if needed).

to maintain symmetric order
and perfect black balance

to fix color invariants

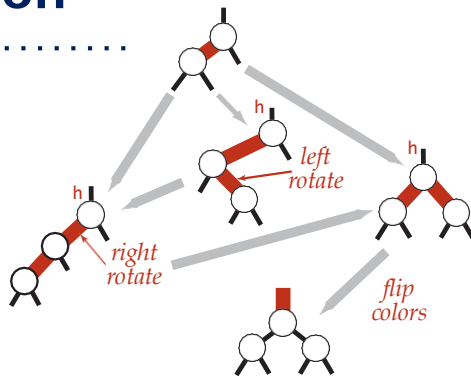
inserting P



Insertion in a LLRB tree: Java Implementation

Same code for all cases.

- Right child red, left child black: **rotate left**.
- Left child, left-left grandchild red: **rotate right**.
- Both children red: **flip colors**.



```
private Node put(Node h, Key key, Value val)
{
```

```
    if (h == null) return new Node(key, val, RED);
```

```
    int cmp = key.compareTo(h.key);
```

```
    if (cmp < 0) h.left = put(h.left, key, val);
```

```
    else if (cmp > 0) h.right = put(h.right, key, val);
```

```
    else if (cmp == 0) h.val = val;
```

```
    if (isRed(h.right) && !isRed(h.left)) h = rotateLeft(h);
```

```
    if (isRed(h.left) && isRed(h.left.left)) h = rotateRight(h);
```

```
    if (isRed(h.left) && isRed(h.right)) flipColors(h);
```

```
    return h;
```

```
}
```

← insert at bottom
(and color it red)

← lean left

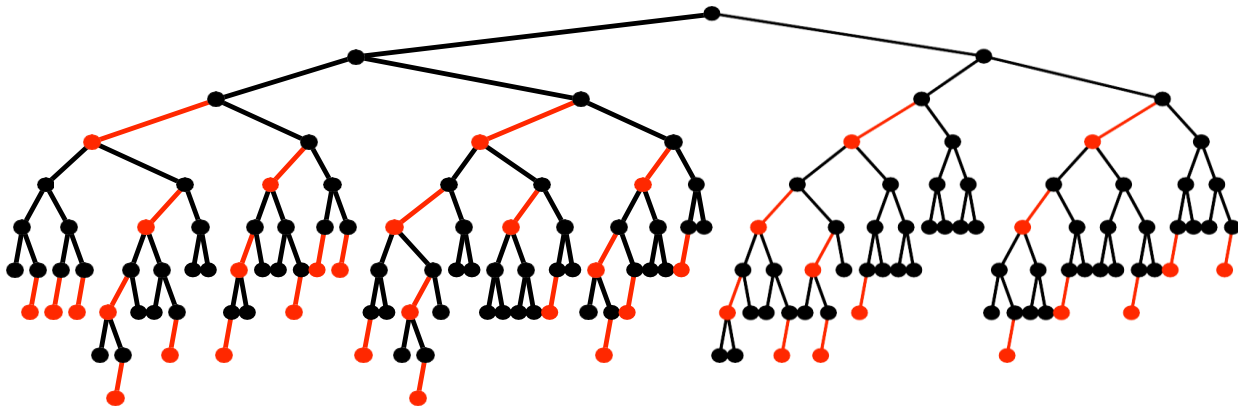
← balance 4-node

← split 4-node

↑
only a few extra lines of code provides near-perfect balance

LLRB Trees Running Time

- Height of tree is $\leq 2 \log_2 N$ in the worst case
- Proof:
 - every path from root to null link has same number of black links
 - never two red links in-a-row



Thank you for your attention !

- In this lecture, you have learned the data structures:
 - Map
 - Binary search trees
 - Balanced search trees
 - 2-3 trees
 - left-leaning red-black BSTs
- Please continue to work on **Exercise 12**